束ねて、考える

*マルチヘッドと **FFN** — 視線を複数系統に、集めた情報に考える場所を*

rkskmt

前回の振り返り

- 言語モデル用の注意ヘッドが完成した：位置埋め込み＋因果マスク付きの **Head**
- でも、このヘッド1本ではまだ GPT は組めない。足りないものが2つ：

> **① クイズ**
>
> 「」の対応を見る視線と、直前の文字を見る視線 — 1枚の「視線の表」で**両方いっぺんに**できるか？

- できない。1枚の表は視線の配分**1通り**しか持てない — 今日の部品1つ目は**視線の複線化**

32文まとめて — バッチの軸

- ミニバッチで学習するので、今日から入力は**複数の文をまとめた3次元**：(バッチ, 文字数, 次元)

Code

```
import torch

x = torch.randn(32, 5, 8)                # 32文・5文字・8次元
print(x.transpose(-2, -1).shape)         # 後ろ2軸だけ入れ替わる
```

Result

```
torch.Size([32, 8, 5])
```

- 部品側の変更は1点だけ：転置を **`k.T`** でなく **`k.transpose(-2, -1)`**（**最後の2軸**だけ入れ替え）と書く
- 行列積 **`@`** も後ろ2軸で走り、バッチの軸は素通し — **32文ぶんの視線の表が一撃で32枚**できる

ヘッドをバッチ対応にする

Code

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class Head(nn.Module):
    def __init__(self, n_embd, head_size, block_size):
        super().__init__()
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer("tril", torch.tril(torch.ones(block_size, block_size)))

    def forward(self, x):
        B, T, C = x.shape
        q, k = self.query(x), self.key(x)
        w = q @ k.transpose(-2, -1) / math.sqrt(k.shape[-1])
        w = w.masked_fill(self.tril[:T, :T] == 0, float("-inf"))
        w = F.softmax(w, dim=-1)
        return w @ self.value(x)

torch.manual_seed(0)
head = Head(n_embd=96, head_size=24, block_size=64)
print(head(torch.randn(2, 10, 96)).shape)
```

Result

```
torch.Size([2, 10, 24])
```

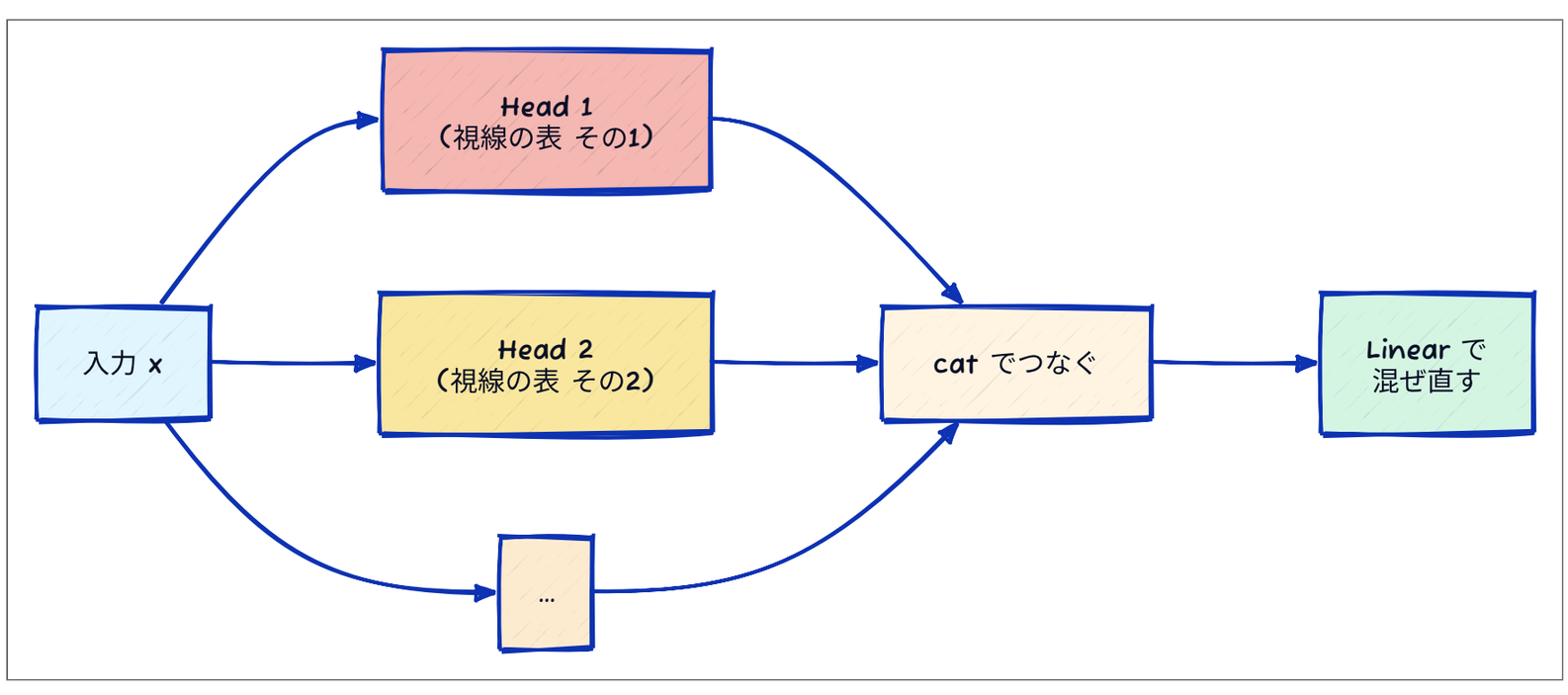
- 前回のクラスと違うのは **`B, T, C = x.shape`** と **`transpose(-2, -1)`** の**2行だけ**。仕組みは同じ

視線は1系統では足りない

- ヘッド1本＝視線の表1枚。でも読むときの視線は1種類ではない

  → 「直前の文字」を見る視線、「文の主語」を見る視線、「開きカッコ」を見る視線……

- 解決は力技：**同じ仕組みを並列に何本も走らせて、結果を横につなぐ**



- どのヘッドがどんな係になるかは決め打ちしない — **学習が勝手に分業を育てる**（訓練回に実物の視線で確かめる）

マルチヘッド注意

Code

```
class MultiHead(nn.Module):
    def __init__(self, n_embd, n_head, block_size):
        super().__init__()
        head_size = n_embd // n_head           # 幅を等分して合計は元どおり
        self.heads = nn.ModuleList(
            Head(n_embd, head_size, block_size) for _ in range(n_head))
        self.proj = nn.Linear(n_embd, n_embd)  # つないだ結果を混ぜ直す

    def forward(self, x):
        return self.proj(torch.cat([h(x) for h in self.heads], dim=-1))

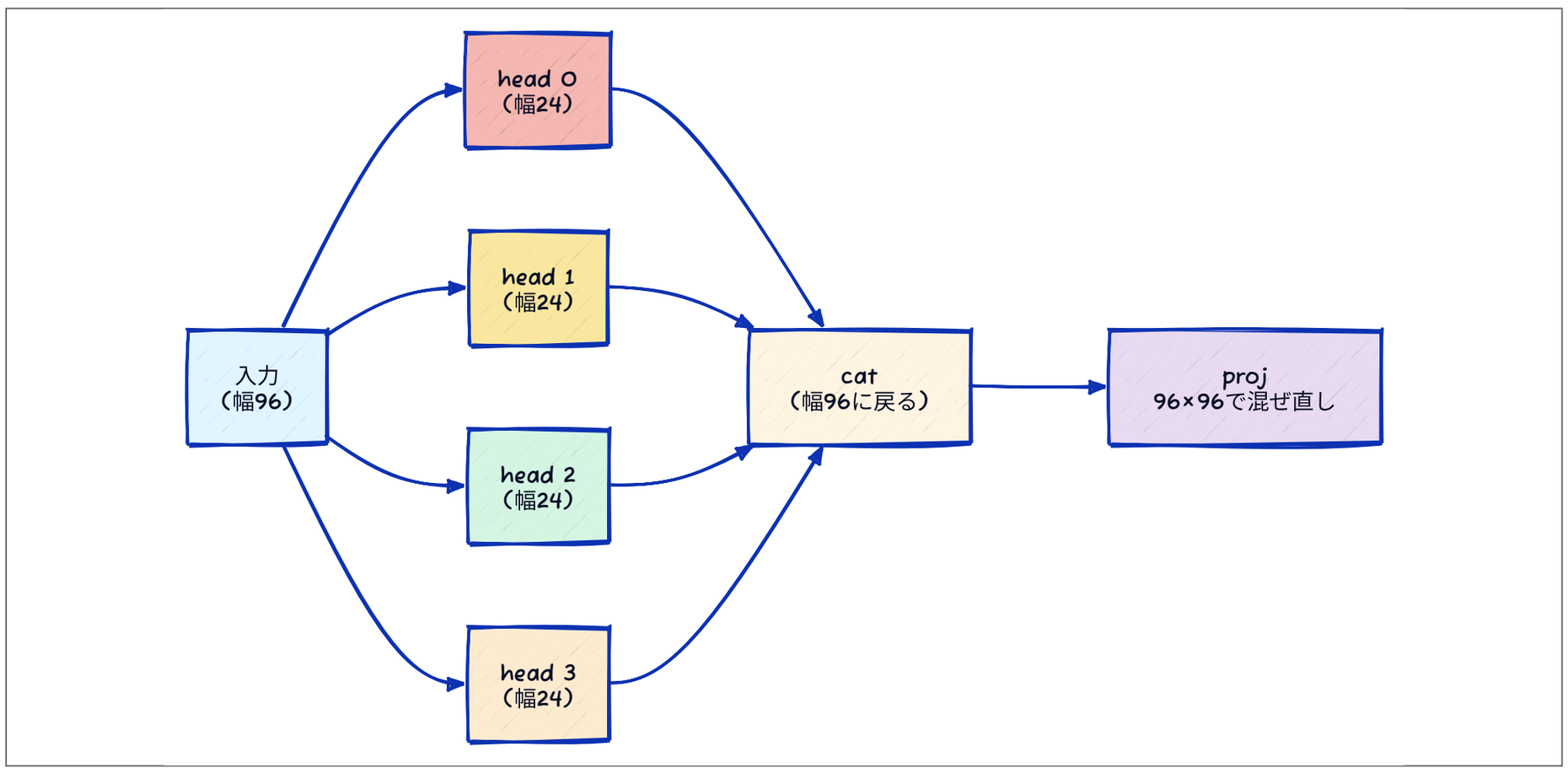
torch.manual_seed(0)
mh = MultiHead(n_embd=96, n_head=4, block_size=64)
x = torch.randn(2, 8, 96)
print(mh(x).shape)                                  # 入口と同じ (2, 8, 96)
```

Result

```
torch.Size([2, 8, 96])
```

- **マルチヘッド注意**（multi-head attention）。新しい数学はゼロ — **`cat`** して **`Linear`** 1枚

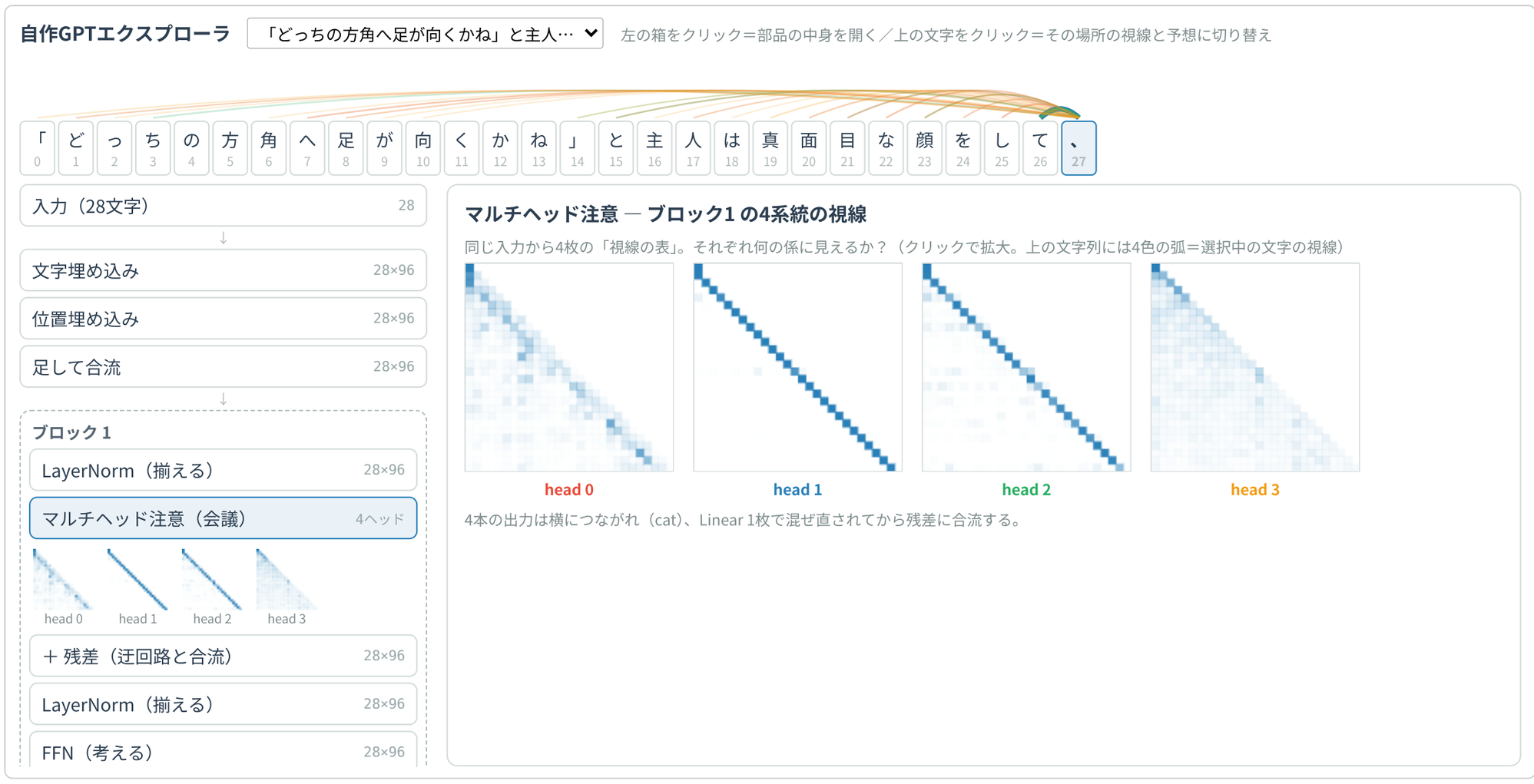
幅の会計 — 等分して、つないで、元通り



- **`head_size = n_embd ÷ n_head`** ＝ 96 ÷ 4 ＝ **24**。幅を等分するから、つなぐと**元の幅に戻る**
- ヘッドを増やしても総コストはほぼ一定 — 増えるのは**視線の系統数**、減るのは**1系統あたりの解像度**（幅）
- 割り切れない頭数（96÷5）だと帳尻が合わずに壊れる — あとの Tweak で実際に壊してみる

4系統の視線を実物で確かめる

訓練済みの完成品（最終回までに組む自作GPT）で、1つのブロックの4ヘッドを並べて見る。



0. 上の文字をクリックして、**4色の弧**（ヘッドごとの視線）が別々の場所へ飛ぶ瞬間を探す
1. 4枚の視線の表それぞれに**係の名前**を付ける（例：「〇〇を見る係」）
2. プリセットをもう1つの文に切り替えて、係の分担が変わらないことを確認

> **ⓘ ノート**
>
> どの係になるかは誰も指示していない — 損失を下げる過程で、4本が勝手に分業した。

エクスプローラを別タブで開く

集めたら、考える — FFN

- 注意は他の文字から情報を**集める**部品。集めた情報を**加工する**場所がまだない
- 各位置ごとに、小さな2層ネット（線形 → ReLU → 線形）を通す

```python
class FeedForward(nn.Module):
    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),   # いったん4倍に広げて
            nn.ReLU(),                       # 折って
            nn.Linear(4 * n_embd, n_embd))   # 戻す

    def forward(self, x):
        return self.net(x)
```



- 注意＝**会議で情報を集める**、FFN＝**自分の席に戻って考える**。この交互が Transformer のリズム
- FFN は位置ごとに独立（隣を見ない）— 見る仕事は注意に任せてある

なぜ、いったん4倍に広げるのか

- FFN の中身は 96 → **384** → 96。広げて、折って、戻す
- 折り紙の勘定（窓を広げるの回でやった折れ線細工）：ReLU の折り目は**中間の幅のぶんだけ**作れる — 1位置あたり**384本**
- 入口と同じ幅のままだと折り目96本 — 「考える広間」は入口より広く取るのが定石で、**4倍は GPT の定番**（GPT-2 も GPT-3 も 4×）

> **ⓘ ノート**
>
> FFN はパラメータの大食らいでもある：$96 \times 384 \times 2 \approx 74{,}000$ 個/ブロック — マルチヘッド注意（約37,000個）の**2倍**。 Transformer の「考える」は、見た目より物量で支えられている。

Tweak　頭数で遊ぶ



- **マルチヘッドのコード** を書き換えて実行（訓練はしないので一瞬）

1	**n_head=4 → 2 や 8**	head_size はいくつになる？ 出力の形が変わらないことを確認
2	**n_head=4 → 5**	エラーで止まる。メッセージを読んで原因を言語化
3	**self.proj(...) → torch.cat(...) をそのまま返す**	形は同じ。では **proj** は何のためにある？

ここで観察

- **n_head=5** は **16x95 と 96x96 は掛けられない** と怒られる — head_size = 96÷5 = 19（切り捨て）で、5本つないでも幅95にしかならないから。**n_embd は n_head で割り切れる値にする**のが約束
- **proj** を外しても形は通る。だが cat 直後は「ヘッド4本の答えが並んでいるだけ」— **ヘッドをまたいで混ぜ直す**のが **proj** の仕事

まとめ

- 入力にバッチの軸が付いた：(バッチ, 文字数, 次元)。部品の変更は **transpose(-2, -1)** だけ
- **マルチヘッド**＝ヘッドを並列に走らせて **cat** → **Linear**。視線が複数系統になり、係の分業は学習が育てる
- **FFN**＝集めた情報を位置ごとに考える2層ネット。「会議」と「自習」の交互
- どちらも**入口と出口が同じ形** — 次はこれを積み重ねる番

現在の設計図　文字＋位置 → **マルチヘッド注意 ⇨ FFN** → 同じ形のベクトル

次回予告

部品はそろった。だが素朴に8段積むと、**勾配が入口に届かず消える**。 深く積むための保険2つを付けて、GPT の全身を組み上げる。

練習問題

以下の問題を**Pythonコード**で解け。

問題1：形を追いかける

`MultiHead(n_embd=96, n_head=4, block_size=64)` に `torch.randn(2, 10, 96)` を入れたとき、次の形を**先に紙で予想してから**検算せよ。

1. 各ヘッドの出力（`h(x)` 1本ぶん）
2. `torch.cat([...], dim=-1)` の直後
3. `proj` を通った最終出力

`forward` に `print` を差し込んで確認すること。

問題2：パラメータの勘定

`MultiHead(n_embd=96, n_head=4, block_size=64)` の**パラメータ総数を手で計算**してから、`sum(p.numel() for p in mh.parameters())` と一致させよ。

1. ヘッド1本：`query`・`key`・`value`（各 96×24、バイアスなし）
2. ヘッド4本ぶんに `proj`（96×96＋バイアス96）を足す
3. 合計をコードと突き合わせ（ズレたらどの部品を数え忘れたか探す）

解答例

▸ 解答例を見る

Result

```
torch.Size([2, 10, 24]) torch.Size([2, 10, 96]) torch.Size([2, 10, 96])
手計算: 36960
実測  : 36960
```

- ヘッド4本で27,648個、`proj` が9,312個 — 「視線の複線化」は意外と安い買い物（GPT 全体82万個の中では脇役）